

提示

软工 ppt 是根据软件生命周期(八个或九个阶段,说法不同)讲的

软件工程

软件工程：软件工程就是用工程化的方法进行软件开发

工程化：任何生产活动都要按照目标化、规范化、文档化、标准化进行。

软件=程序+数据+文档 +服务

软件特性：复杂性、易变形、不可见性、服从性、非连续性

软件危机：

表现：ppt 2 11-15 了解



软件危机概念：在计算机软件的开发和维护过程中所遇到的一系列严重问题。

原因：

【软件开发成本估计不准,开发成本超出预算】

【开发进度不能保证,交付时间一再拖延】

【开发出来的产品不符合用户的需求】

→ 忽视软件开发前期的调研和分析工作

【软件产品质量无法保证】

→

没有统一的、规范的方法论指导
忽视测试阶段的工作

【软件的可维护程度低】

→

文档资料不齐全，忽视人与人的交流
忽视测试阶段的工作
忽视软件的维护

【软件开发生产率的发展跟不上硬件发展速度和人们需求的增长】

→

没有统一的、规范的方法论指导

软件工程：

概念：软件工程就是用工程化的方法进行软件开发

发展：

三要素：过程、方法、工具

过程：支持软件开发各个环节的控制和管理

方法：完成软件开发任务的 技术手段

工具：为软件开发方法提供自动的或半自动的软件支撑环境

软件生命周期：

概念：软件从功能确定、设计到开发成功投入使用，并在使用中不断地修改、增补和完善，直到被新的需要所代替而停止使用该软件的全过程。

任务：软件定义、需求开发、软件设计、软件构造、软件测试、软件维护

软件定义、软件开发、运行维护（8 个阶段）：1 问题定义 2 可行性研究 3 需求分析

4 总体设计 5 详细设计 6 实现 7 测试 (单元测试和综合测试)8 运行维护

🌀 什么是软件过程？

软件过程(Software Procedure)是为获得软件产品，在软件工具支持下由软件工程师完成的一系列软件工程师活动。

软件过程模型：

1.瀑布模型：适合于用户需求明确、完整、无重大变化的软件项目开发

瀑布模型的成功在很大程度上是由于它基本上是一种文档驱动模型，同时也是他的一个主要缺点。

瀑布模型的优点：有利于大型软件开发过程中的人员的组织，管理，有利于软件开发方法和工具的研究，从而提高了大型软件项目开发的质量和效率。

瀑布模型的缺点：(1)开发过程一般不能逆转，否则代价太大(2)实际的项目开发很难严格按照该模型进行

2.原型模型：主要用于不确定的软件开发 类型：抛弃型、演化型

在用户不能给出完整、准确的需求说明，或者开发者不能确定算法的有效性、操作系统的适应性或人机交互的形式等许多情况下，可以根据用户的一组基本需求，快速建造一个原型。

原型模型的优点：(1)可视化，强化沟通，降低风险，节省后期变更成本，提高项目成功率。(2)实际的软件在充分考虑了质量和可维护性之后才被开发。

原型模型的缺点：(1)没有考虑软件的总体质量和长期的可维护性。(2)为了演示，可能采用不合适的操作系统，编程语言，效率低的算法，进而构成了系统的组成部分。

3.增量模型：是**渐进地开发逐步完善**的软件版本的模型。 **不断应用瀑布模型和原型模型。**

增量模型的优点：(1)产品被分解成**多个增量构件**，开发人员可逐步开发(2)在较短的时间内向用户提交**可完成部分工作的产品**(3)**以组件为单位**进行开发降低了软件开发的**风险**。(4)**开发顺序灵活**

增量模型的缺点：(1)软件体系结构必须是开放的(2)需要精心的设计

4.螺旋模型：**降低风险**，适用于**规模大的复杂系统**。 将**瀑布模型**和**增量模型**加入**风险分析**

螺旋模型的开发过程的四个阶段：**制定计划**，**风险分析**，**实施工程**和**客户评估**

螺旋模型的优点：(1)设计上灵活性，可以在项目的各个阶段进行变更(2)以小的分段来构建大型系统，使成本计算变得容易

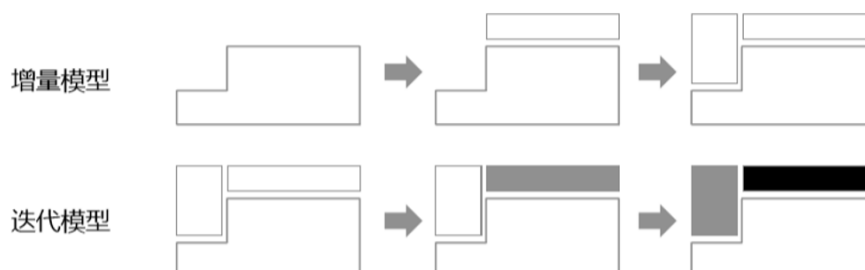
螺旋模型的缺点：(1)风险驱动必须靠谱，否则风险更大(2)迭代次数增加，工作量加大，软件开发成本增加(3)费用昂贵，只适合大型软件的开发

5.喷泉模型：**自下而上**，**用户需求为动力**，支持**面向对象**

迭代开发：整个项目被拆分成一系列小的迭代。通常一个迭代的时间都是固定的，不会太长，例如 2-4 周。每次迭代只实现一部分功能，做能在这个周期内完成的功能。

增量模型：在每一个新的发布中逐步增加功能直到构造全部功能。

迭代模型：一开始提交一个完整系统，在后续发布中补充完善各子系统功能。



RUP 过程模型（统一过程模型）：

开发生命周期(四个阶段、九个核心工作流)：

四个阶段：初始阶段、细化阶段、构造阶段、移交阶段

九个核心工作流：

6 个核心过程工作流：业务建模、需求、分析与设计、实现、测试、部署，

3 个核心支持工作流：配置与变更管理、项目管理、环境

特点：RUP 是风险驱动的、基于 UseCase 技术的、以架构为中心的、迭代的、可配置的软件开发流程。我们可以针对 RUP 所规定出的流程，进行客户化定制，定制出适合自己组织的实用的软件流程。因此 RUP 是一个流程定义平台，是一个流程框架。

敏捷开发：

特点：以人为核心、循环迭代、响应变化

四条基本价值观：个体和交互、可以工作的软件、客户合作、响应变化

开发模型的特点：

Scrum：有明确的目标、高度自主权，紧密的沟通合作，高度的弹性迎接各种变化和挑战，确保每个阶段都朝着目标推进。

极限编程(XP)，是一种“轻量型”的以编码为核心任务的灵活软件开发方法。敏捷开发中有名的一个

XP 目标：在最短的时间内将较为模糊、变化较大的用户需求转化为符合用户要求的软件产品。

开发流程：

XP 软件开发过程：聆听、测试、编码、设计的迭代循环过程

Scrum 和极限编程的特点：scrum 偏重项目管理、XP 偏重项目实践

RUP（统一过程模型）与 XP（极限编程）：

- 1.RUP 面向管理层面，XP 面向实施层面。两者是相辅相成、互相补充的。
- 2.在 RUP 管理框架之下，才能在迭代实施过程中采用 XP，发挥 XP 的“快速”作用。

几种过程模型的对比：

序号	模型名称	优点	缺点	适用范围
1	瀑布模型	简单好学	逆转性差	需求不发生或变化很小
2	快速原型模型	开发速度快	不利于创新	已有产品的原型
3	增量模型	可以分阶段提交	有时用户不同意	系统可拆卸和组装
4	螺旋模型	将风险分析拓展到各个阶段中	建设周期长	庞大、复杂、高风险项目
5	喷泉模型	提高开发效率	不利于项目的管理	面向对象开发
6	统一过程模型	需求可变	风险大	有高素质软件团队
7	敏捷模型	提高开发效率	不适合大团队、大项目	小团队，小项目

项目计划阶段：

问题定义、可行性研究、项目计划

问题定义目的：弄清软件的性质、规模和限制条件，写出问题定义报告。

可行性研究：从技术、经济和法律角度弄清该软件是否值得开发，写出可行性分析报告。结论：可行、基本可行、不可行

项目计划：确定要开发之后，对项目进度、所需资源进行规划，写出项目开发计划书。

软件项目分为两种类型：**委托式加工**和**开发市售产品**

1. 以下关于系统原型的叙述中，不正确的是（ ）。

- ☐ A 可以帮助导出系统需求并验证需求的有效性
- ☐ B 可以用来探索特殊的软件解决方案
- ☒ C 可以用来指导代码优化
- ☐ D 可以用来支持用户界面设计

2. 以下关于增量模型的叙述中，不正确的是（ ）。

- ☐ A 核心的产品往往首先开发，需经历充分的“测试”
- ☐ B 第一个可交付版本所需要的成本低，时间少
- ☐ C 即使一开始用户需求不清晰，对开发进度和质量也没有影响
- ☒ D 适用于大型软件项目的开发，对开发人员要求不高

3. 以下叙述中，（ ）不是一个风险。

- ☐ A 由另一个小组开发的子系统可能推迟交付，导致系统不能按时交付客户
- ☒ B 客户不清楚想要开发什么样的软件，因此开发小组开发原型帮助其确定需求
- ☐ C 开发团队可能没有正确理解客户的需求
- ☐ D 开发团队核心成员可能在系统开发过程中离职

4. 某企业拟开发一个企业信息管理系统，系统功能与多个部门的业务相关。现希望该系统能够尽快投入使用，系统功能可以在使用过程中不断改善。则最适宜采用的软件过程模型为（ ）。

- ☐ A 瀑布模型
- ☐ B 原型模型
- ☒ C 演化（迭代）模型
- ☐ D 螺旋模型

5. 关于RUP模型的特点，下面叙述不正确的为（ ）。

- ☒ A 开发过程分为分析、细化、构造、交付四个阶段
- ☐ B 每个阶段围绕九个核心工作流分别迭代
- ☐ C 基于风险驱动，以用例技术、架构设计为中心
- ☐ D 更合理的生命周期模型，可灵活定制

6. 下面关于敏捷开发的叙述，不正确的为（ ）。

- ☐ A 敏捷方法以人为本，依靠程序员的自我管理提高效率
- ☒ B 可以工作的软件和全面的文档是首要的进度度量标准
- ☐ C 欢迎需求变化，并利用这种变化来提高用户的竞争优势
- ☐ D 团队提倡面对面交谈，业务人员和开发人员必须天天在一起工作

7. 下面关于Scrum开发方法的叙述，不正确的为（ ）。

- ☐ A Scrum方法偏重于项目管理
- ☐ B 将软件开发周期分为多个小的迭代周期
- ☒ C 一个迭代周期针对一个或多个用户故事
- ☐ D 每个迭代周期中不允许需求发生变化

认识到需求分析的重要性:

软件开发是用户与开发者共同参与的过程，项目涉众人员之间必须经过充分交流；

用户与开发者的知识领域不同，缺乏共同语言，存在对问题理解的歧义；

软件开发失败的原因大约超过50%是需求不合理而急于编程引起的；

在需求分析阶段的错误会引起错误的放大，后期发现错误时要花费更大的精力修改。

软件过程模型：传统模型、RUP 模型、敏捷模型（scrum 和 Xp Xp 重要）

了解需求分析阶段的工作任务：系统必须做什么

了解 内容：需求是系统为满足客户期望的目标而完成的行为，要体现出对问题领域的清晰理解，给出系统的使用场景和上下文。

过程：

需求不同层次：业务需求、用户需求、功能需求（系统需求） 三个写前三个

ppt5.25

需求工程：需求开发、需求管理

获取需求的方法：

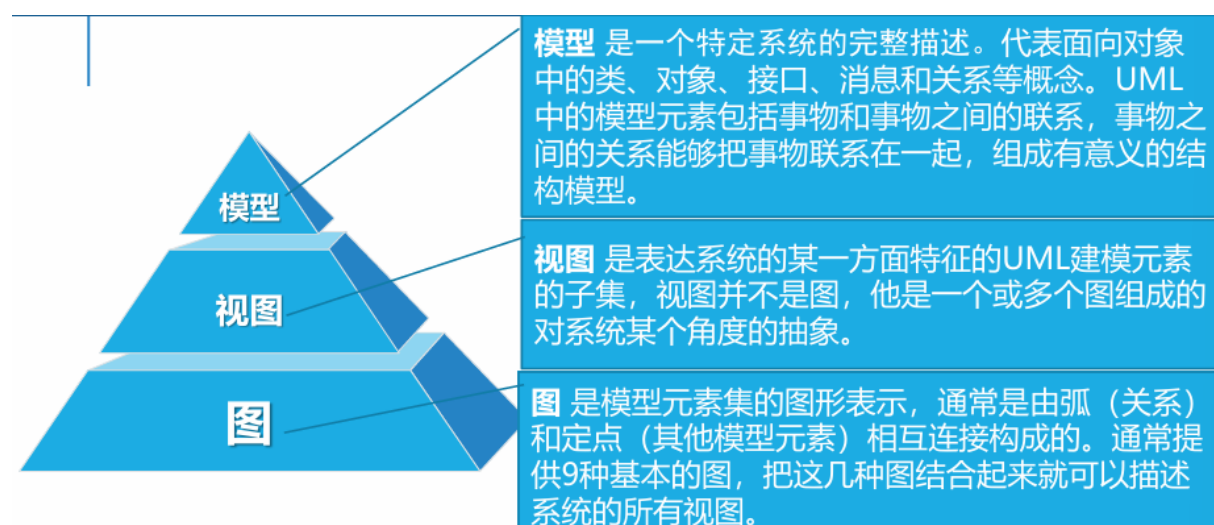
第一阶段：访谈式、第二阶段：诱导式、第三阶段：确认式

需求建模方法：结构化分析建模 SA、面向对象分析建模 OOA

需求工程过程：需求获取、分析需求、文档编写、需求验证、需求变更

UML (统一建模语言) 语言概述：它不仅统一了 Booch、Rumbaugh 和 Jacobson 的表示方法，而且对其做了进一步的发展，并最终统一为大众所接受的标准建模语言。

UML 的组成：



建立用例模型：1 定义系统 2 确定执行者和用例 3 描述执行者和用例关系 4 确认模型

UML 五类图：用例图、静态图（类图、对象图和包图）、行为图（状态图和活动图）、交互图（顺序图和协作图）、实现图（组件图和配置图）

系统用例建模：

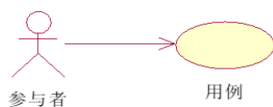
用例描述：简要描述、前置条件、基本事件流、其他事件流、后置条件等

用例图举例 谁指向谁看清 会分析 能找到用户,会分析时间

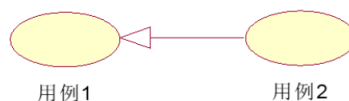


用例图中的关系（难点）

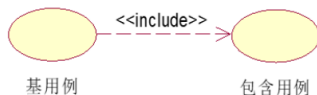
用例图中有以下几种关系：



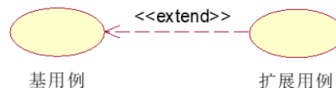
1. 关联



2. 泛化



3. 包含



4. 扩展

包含关系 (Include)： 指一个用例作为另一个用例必需的部分被使用，是依赖关系的一种。被包含的叫**包含用例**，它是**基础用例(base use case)**中一个**不得不执行**的用例部分。

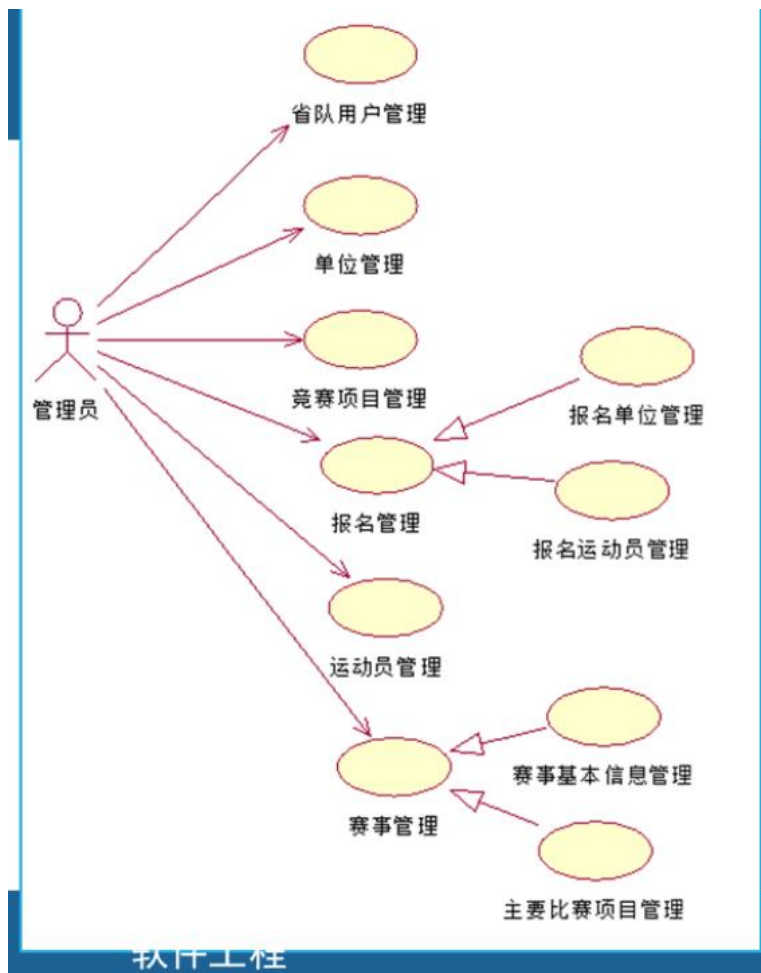
扩展关系 (Extend)： 一个用例也可以被定义为基础用例的增量扩展，这称作扩展关系，也是依赖关系的一种。即可以将一个功能完整的基础用例进行扩展，使其能够完成一个新的功能。



网上报名系统的业务用例建模

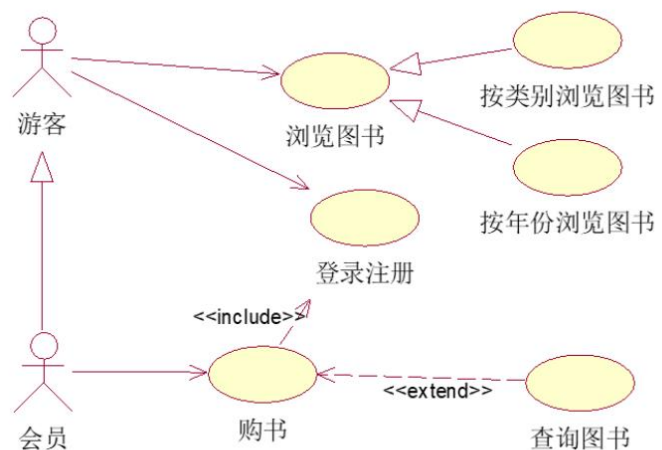
3. 中国赛艇协会管理人员的主要业务：

- | | |
|-------------|------------|
| (1) 各省队用户管理 | (5) 报名管理 |
| (2) 单位管理 | i) 报名单位 |
| (3) 运动员管理 | ii) 报名运动员 |
| (4) 竞赛项目管理 | (6) 赛事管理 |
| | i) 赛事基本信息 |
| | ii) 主要比赛项目 |



操作者之间可以存在泛化关系，类似的参与者可以组成一个层级结构。

在网上书店的例子中，会员是游客的泛化，游客有浏览图书的用例，而会员不仅包含游客的全部用例，还具有自己特有的购买图书用例，参见右图。



用例说明

用例编号	ORS 003
用例名	新增运动员报名
用例描述	省队用户填写报名运动员信息提交报名信息
参与者	省队用户
前置条件	浏览赛事信息点击报名
基本路径	1. 点击“新增人员报名” 2. 输入参赛人员信息（姓名，性别，年龄，赛事名称，参赛单位，运动员注册号，竞赛项目名称，备注） 3. 点击“报名提交” 4. 显示已有报名人信息，报名成功。
其他路径	无
异常事件	无
后置条件	无

活动图

软件开发方法发展：面向过程、面向对象、面向组件、面向服务、面向云计算

结构化软件工程方法(面向过程)

结构化需求分析（SA）：[数据流图](#)、[数据字典](#)

结构化设计(SD):[模块结构图](#)、[程序流程图](#)

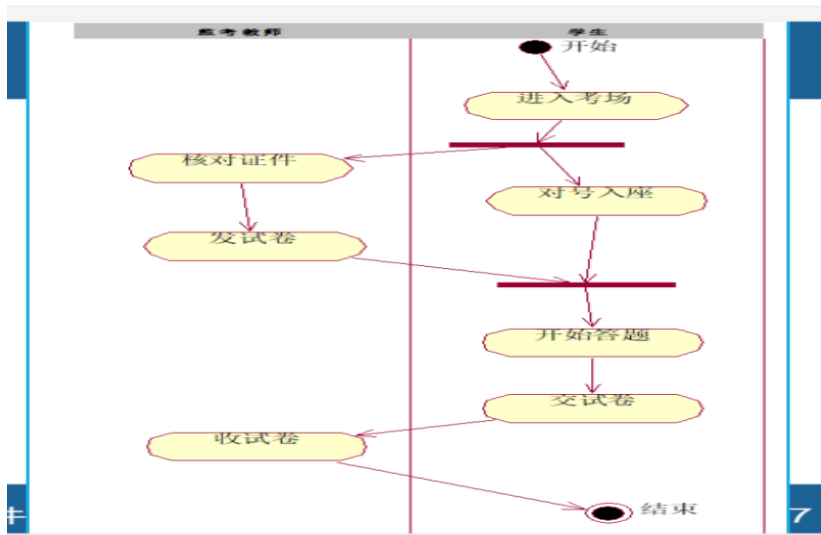
结构化编程、结构化测测、结构化维护

面向对象软件工程方法：

面向对象分析 OOA、面向对象设计 OOD、面向对象编程 OOP、面向对象测试

OOT、面向对象维护

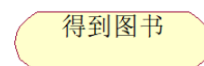
面向对象建模：[功能模型](#)、[对象模型](#)、[动态模型](#)



活动图的组成元素

- **活动图**的元素包括：初始节点、终点、活动节点、转换、分支、分岔与汇合。
- 其中，转换、分支、分岔与汇合把多个活动节点连接在一起。

动作状态



动作流



分支（判定条件）



分叉和汇合
(表示并发和同步)



泳道



并发活动：分岔和汇合

泳道：表示各个活动由谁负责的信息

面向对象方法学：

面向对象的概念：

类和对象 属性和方法 抽象和封装 消息传递 继承和多态、重载

面向对象分析：

- ④ 面向对象分析，就是抽取和整理用户需求并建立问题域精确模型的过程。
- ④ 面向对象分析过程从分析陈述用户需求的文件开始。
- ④ 接下来，系统分析员应该深入理解用户需求，抽象出目标系统的本质属性，并用模型准确地表示出来。
- ④ 在面向对象建模的过程中，系统分析员必须认真向领域专家学习。
- ④ 在面向对象建模的过程中，还应该仔细研究以前针对相同的或类似的问题域进行面向对象分析所得到的结果（可重用）。

第九次 类图

看看类图怎么画

分类：领域类图、实现类图

类：名称、属性、操作 接口：名称 方法

类的抽象层次：概念车、说明层和实现层

关系：依赖、泛化、实现、关联

④ 各种关系的强弱顺序：

泛化 == 实现 > 组合 > 聚合 > 关联 > 依赖

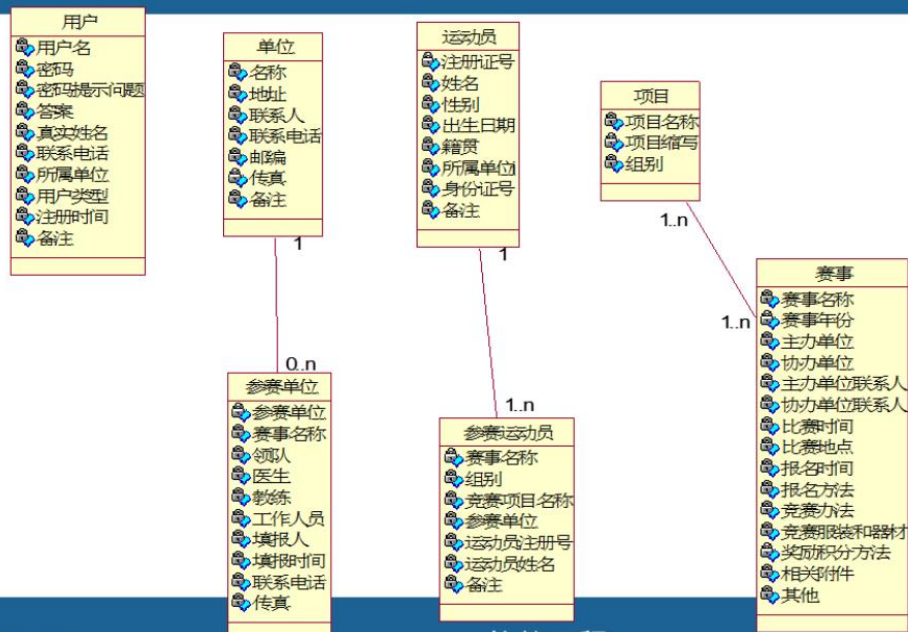
特殊关联关系：聚合关系、合成关系

不同层次的类：实体类、控制类、边界类

领域模型的三大模型：视图模型、逻辑模型、实体模型



网上报名系统的领域建模



2023/4/11

软件工程

6

需求建模:

需求类型: 非功能性需求和功能性需求

需求规格说明书(不清楚,稍微看看)

获取非功能需求:

几个方面:

可靠性: 安全性、事务性 ACID、稳定性

可用性:

2 可用性

容易学习

→ 客户需要多长时间来掌握软件的使用？

使用效率

→ 客户需要多长时间、执行多少次操作来完成一个关键任务？

记忆性

→ 当客户离开再次回来时，他的工作是否能够被记忆下来以便继续执行？

错误恢复

→ 当系统出现故障时，客户是否能从故障中恢复他已经完成的工作？

主观满意度

→ 客户在使用软件过程中是否感到愉悦？

有效性：性能、可伸缩性、可扩展性

可移植性：

通常可移植性都是软件针对平台而言的。软件应该能够适应更多种平台。

在软件开发过程中，应当使用成熟的，公开支持的标准，尤其是大厂商所支持的标准，来确保系统不丧失软件环境的可移植性。

第十次 数据库设计

软件设计两部分：总体(结构,概要)设计和详细设计

总体设计：在需求规格说明书的基础上描述系统的架构、功能模块的划分、模块接口的定义、用户界面设计、数据库设计 严谨编写数据字典

详细设计：在需求规格说明书的基础上描述系统的架构、功能模块的划分、模块接口的定义、用户界面设计、数据库设计

数据的持久化：指要永久使用的数据或系统产生的数据如何保持

常见保存方式：数据库管理系统和文件存储模式

系统设计

系统设计阶段的主要任务 数据库设计

系统的静态结构设计



软件体系结构的发展阶段

- 主机/终端 (Host/Terminal, H/T)
- 客户机/服务器 (Client/Server, C/S)
- 浏览器/服务器 (Browser/Server, B/S)
- 多层体系结构
- 面向服务的体系结构
- 面向 workflow 引擎

简答题可能会考

软件架构提供了**组件和容器**

体系结构相关知识

MVC 设计模式：模型 (model) 对象、视图对象 (View) 、控制器对象 (Control)

DAO 设计模式：功能：**1 封装数据源 2 封装对数据库的操作**

系统包图：包的可见性：**共有访问、私有访问、保护访问**

第十一次

一个构件就是程序的一个功能要素，也称为模块。包括：**程序的处理逻辑、需要的内部数据、保证构件被调用和实现数据传递的接口。**

软件设计的基本原理和相关概念

模块化：软件系统越来越复杂，需要将其划分为若干部分分而治之。

(逐步求精)

逐步求精是自顶向下、逐层分解、分而治之来解决复杂问题的思想方法。

抽象：为了降低问题的复杂度，以得到问题领域中较为简单的概念。

支持**过程抽象**和**数据抽象**

过程抽象：把用文字形成的功能描述演变成代码。

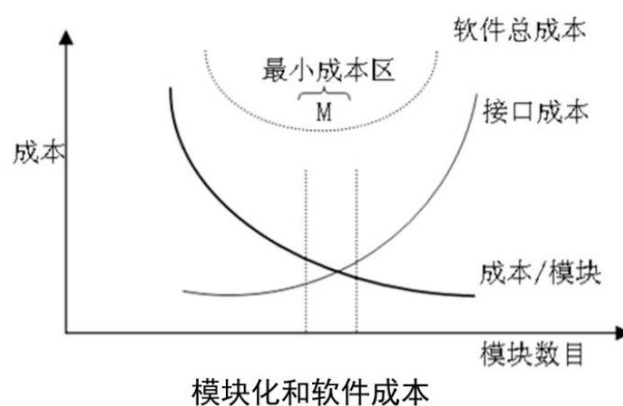
数据抽象：文字表达的数据结构演变成数据库表和存储过程。数据抽象与过程抽象一样，能使设计者按不同的详细程度表示数据对象。

信息隐蔽和局部化：为了避免某个模块的行为干扰同一系统中的其他模块，把必要的信息隐藏起来。

信息隐藏是实现**抽象**、**模块化**机制的基本支撑。

模块独立：每个模块只完成特定的单一功能，不同模块之间的联系尽可能少，尽可能减少公共变量和数据结构，有助于系统维护及软件的复用。

- 模块数增加时，模块间的关系也随之增加，接口和集成的工作量也随之增加。
- **结论：寻找最佳模块化程度平衡点。**



两个定性标准度量：**耦合度**和**内聚度**

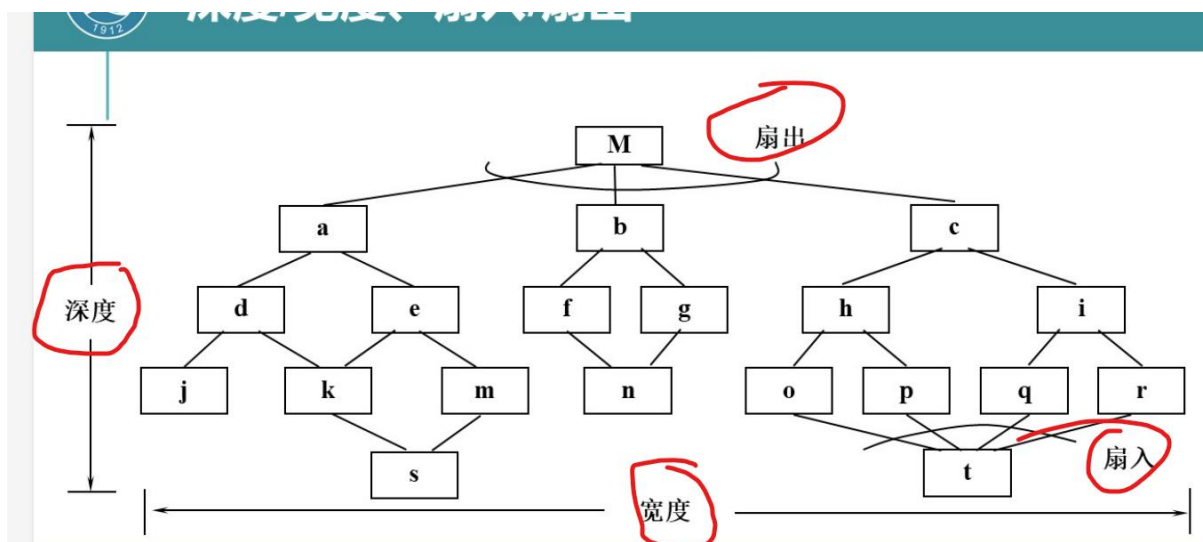
独立性标准：高内聚、低耦合

耦合度七个等级：① 非直接耦合 ② 数据耦合 ③ 特征耦合 ④ 控制耦合 ⑤ 外部耦合 ⑥ 公共耦合 ⑦ 内容耦合

三种内聚：服务内聚、类内聚、一般-特殊内聚

内聚七个等级：偶然性内聚 逻辑性内聚 时间性内聚 过程性内聚 通讯性内聚 顺序性内聚 功能性内聚

深度/宽度、扇入/扇出 深度控制层数、宽带控制跨度、扇出数 直接控制其他模块数
(依赖性) 扇入数 能直接控制的上级模块数 (复用性)



深度/宽度、扇入/扇出

- 一个软件结构的深度(depth) — 表示控制的层数
- 一个软件结构的宽度(width) — 表示控制的跨度
- 一个模块的“扇出数” (fan out) — 指该模块直接控制的其他模块数(依赖性)
- 一个模块的“扇入数” (fan in) — 指能直接控制该模块的上级模块数(复用性)

软件重用：是指同一事物不作修改或稍加改动就多次重复使用。

层次:知识重用、方法和标准的重用、软件成分的重用

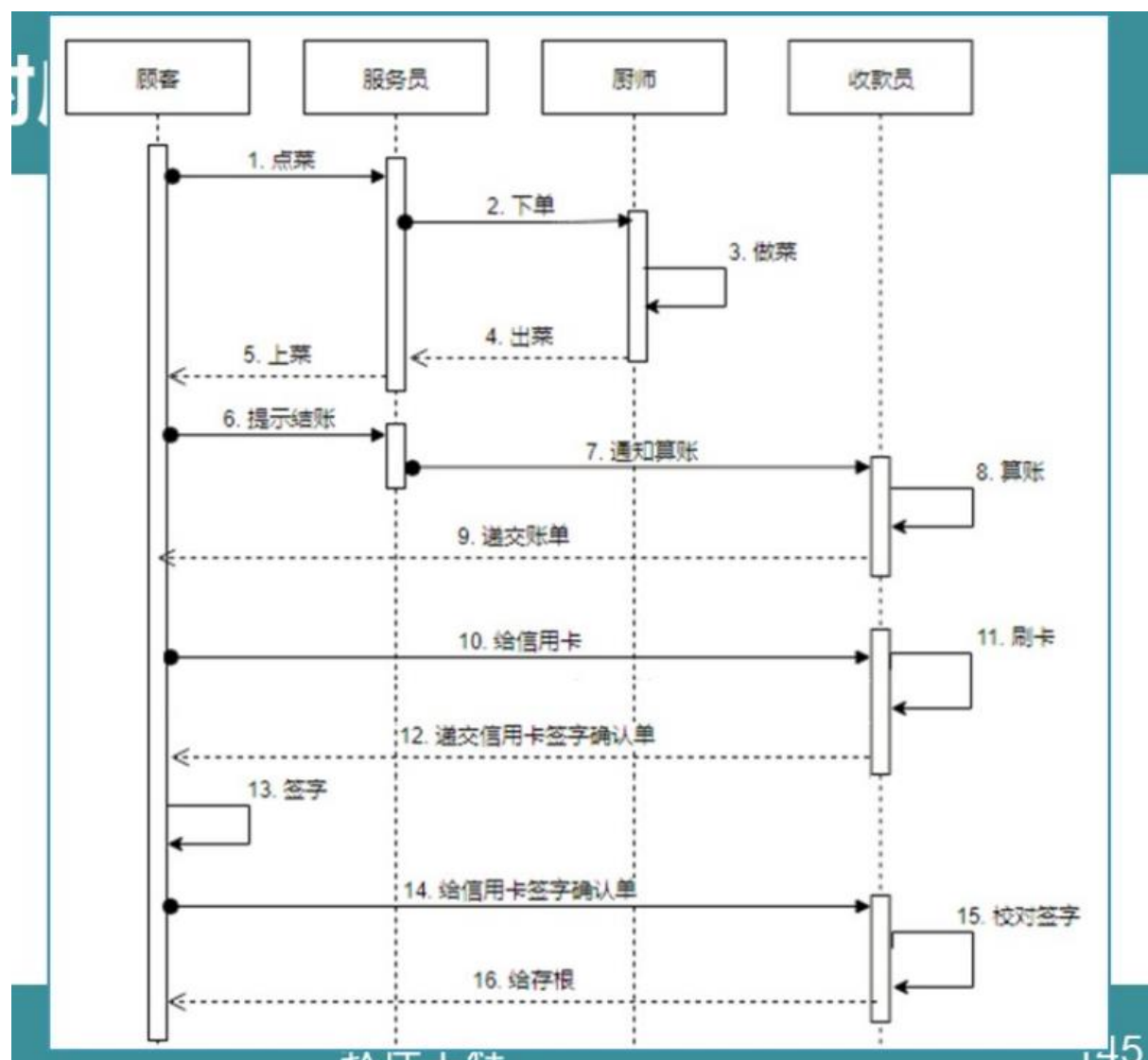
软件成分重用三个级别：1 几种形式的代码重用、2 设计结果重用、3 分析结果重用

面向对象设计原则：抽象，信息隐蔽，模块独立，高内聚，低耦合 主要前五个原则

单一责任原则、依赖倒置原则、里氏替换原则、开放封闭原则、接口隔离原则、合成复用原则、迪米特法原则。

链是对象间的发送消息的路径

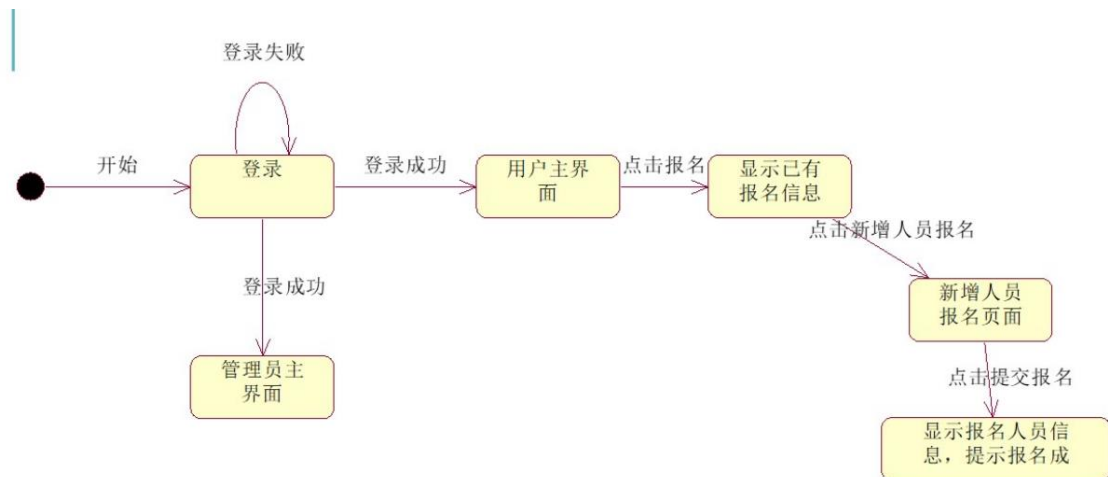
时序图



协作图没让画,可以瞅一眼

十二课 状态图

状态图：状态和转换



建立系统状态模型

状态图建模技术：使用状态图一般是对系统中反映型对象建模，特别是对类、用例和系统实例的行为建模。

系统交互界面设计

人机界面设计原则：

- 1 让用户拥有控制权
- 2 减少用户的记忆负担
- 3 保持界面一致

第十三次

软件的详细设计

详细设计的工作任务

1. 界面的详细设计是要根据原型设计界面的元素和交互功能，用算法表达界面的操作逻辑。
2. 为每个模块确定采用的算法，选择某种适当的工具表达算法的过程，写出模块的详细过程性描述。
3. 确定每一模块使用的数据结构，为以后的编写程序做好充分的准备。
4. 确定模块接口的细节。
5. 对数据库进行物理结构设计，建立表结构、关联、完整性、建立索引和视图等。

面向对象的详细设计：属性的数据结构设计、方法的设计、实现接口所需机制的设计

详细设计的工具

程序流程图（重要） 盒图（N-S图）（看懂） PAD图（了解） 判定表（会用） 判定树（会用）

软件的编码实现

为了执行规范，每个软件开发人员必须一致遵守编码规范

编写高质量代码和编码标准规范 代码审查和结对编程

第十四次

软件系统的测试

四个方面理解软件测试：

软件测试贯穿软件开发的整个生命周期

测试是为了证明程序有错，而不是证明程序无错误。

测试包含了“分析”或“运行”软件。

软件测试有两个基本功能：验证和确认

测试用例设计的内容：正确数据、错误数据、边界数据。

单元测试、集成测试、系统测试、验收测试

软件测试的概念：软件测试，是为了发现错误而执行程序的过程。

软件测试的方法：

静态方法:通过人工分析或程序正确性证明的方式来确认程序正确性。

人工测试方法、计算机辅助静态分析方法 静态方法

动态方法:通过输入数据，运行软件，观察执行状态和结果，来检验软件的动态行为和运行结果的正确性

白盒测试方法、黑盒测试方法 动态方法 动态测试的两个基本要素：被测试程序、测试数据

测试用例的三个要素：前提条件和操作步骤、预期结果、实际结果

如果知道产品的内部工作过程，可以通过测试来检验产品内部动作是否按照规格说明书的规定正常进行 ---- 称为白盒测试。

动态测试技术

1、白盒测试
(White Box Testing)

2、黑盒测试
(Black Box Testing)

如果已经知道了产品应该具有的功能，可以通过测试来检验是否每个功能都能正常使用 ---- 称为黑盒测试。

白盒测试技术：

逻辑覆盖：是以程序内部的逻辑结构为基础的设计测试用例的技术

五种逻辑测试标准 57 页看一下 知道分别是什么意思

语句测试、判断测试、条件测试、判断覆盖、判断/条件覆盖、条件组合覆盖

发现错误的 能力	标 准	含 义
1(弱)	语句覆盖	每条语句至少执行一次
2	判定覆盖	每一判定的每个分支至少执行一次
3	条件覆盖	每一判定中的每个条件，分别按“真”、“假”至少各执行一次
4	判定/条件覆盖	同时满足判定覆盖和条件覆盖的要求
5 (强)	条件组合覆盖	求出判定中所有条件的各种可能组合值，每一可能的条件组合至少执行一次

基本覆盖法：点覆盖、路径覆盖 65 页 自己看一下知道什么是基本覆盖

黑盒测试

黑盒测试技术：**等价划分法**、边界值分析法、错误推测法、因果图法、决策表法、场景法等

如何划分等价类：**有效等价类、无效等价类** 标准：覆盖、不相交、代表性

等价类划分举例

输入条件 (等价类)	有效等价类	无效等价类
长度	(1) 等于8位	(2) 大于8位 (3) 小于8位
字符集合	(4) 全部为数字	(5) 含有非数字字符
规则	(6) 不全为零	(7) 8位全为零

边界值分析法：**测试内点和测试外点**

编号	等价类	测试说明	测试数据	期望结果
1	长度	仅有1个合法字符	2	提示位数不足
2		比有效长度少1	1234567	提示位数不足
3		类型及长度均有效	12345678	输入有效
4		比有效长度多1	123456789	提示位数超出
5	字符集合	不足8位1个非法字符	d	提示有非法字符
6		8位只有1个非数字字符	123D4567	提示有非法字符
7		全部是非数字字符	sb_%&cbq	提示有非法字符
8	规则	只有一个0	0	提示位数不足
9		末尾有1个数字字符	00000001	输入有效
10		开头有1个数字字符	10000000	输入有效
11		全部是0数字字符	00000000	提示全0非法输入

黑盒白盒比较

	黑盒测试	白盒测试
优点	适用于各阶段测试； 从产品功能角度测试； 易入手生成测试数据。	可构成测试数据使特定程序部分得到测试； 有一定的充分性度量手段； 可有较多工具支持。
缺点	某些代码得不到测试； 如果规格说明有误，则无法发现； 易进行充分性测试。	通常不易生成测试数据； 无法对未实现规格说明的部分进行测试； 工作量大，通常只用于单元测试，有应用局限。
性质	是一种 确认 技术，回答“我们在构造的是一个正确的系统吗？”	是一种 验证 技术，回答“我们是在正确地构造一个系统吗？”

软件测试的过程和类型:

单元测试： 额外建立两种模块：桩模块、驱动模块

集成测试： 增量集成法和非增量集成法

系统测试，验收测试：

测试阶段	主要依据	测试人员 测试方式	主要测试内容
单元测试	详细设计文档	由程序员执行黑盒+白盒测试	接口测试、边界条件测试、局部数据结构与算法测试、路径测试、错误处理路径的测试
集成测试	概要设计文档和软件需求	由程序员执行黑盒测试	接口测试、路径测试、功能测试、性能测试
系统测试	软件需求	由独立测试小组执行黑盒测试	功能测试、健壮性测试、性能测试、UI测试、安全性测试、压力测试、可靠性测试、安装/反安装测试等
验收测试	软件需求	由用户执行黑盒测试	

面向对象测试策略：类内测试、类间测试、基于场景的测试

白盒测试和黑盒测试是软件测试的两类基本方法

测试包含静态测试和动态测试,测试步骤至少分为:

模块测试 --- 单元 子系统测试 --- 局部 系统测试 --- 集成 验收测试

--- 用户参与

第十五次

调试就是把症状和原因联系起来的尚未被人深入认识的智力过程。

软件的调试的方法：蛮干法、回溯法、原因排除法

软件维护的类型与过程：

维护的类型：改正性维护、适应性维护、扩充与完善性维护、预防性维护

软件维护过程：维护就是修改软件；另一个关键词是“过程”，过程暗喻不是单一的活动，是活动的集合和活动的有序组合，过程就需要管理。

文档类型：用户文档和系统文档

软件再工程过：预防性维护又称为软件再工程。指的是重新构造或编写现有系统的一部分或全部，但不改变其功能。

重构的概念：就是在不改变软件现有功能的基础上，通过调整程序代码改善软件的质量、性能，使其程序的设计模式和架构更趋合理，提高软件的扩展性和维护性。

重构与增强不同时进行 戴一顶帽子只做事

软件维护的 4 类活动：（改正性、适应性、完善性、预防性）决定软件可维护性的

基本要素：（可理解、可测试、可修改、可移植和可重用性）文档是影响软件可维护性的决定因素